Low-Code Experiment

Case Study on the Limitations of Low-Code Platforms for a Complex Enterprise Use Case

Sachitha S
241501171
AIML

Abstract

This case study analyses where low-code platforms such as Zoho Creator fall short when applied to a complex, large-scale enterprise use case. A fictional hotel chain tries to extend a successful single-hotel booking application into a group-wide central reservation and revenue management system. The study identifies the limitations that appear in performance, concurrency, algorithms, integration, data volume, portability and governance, and recommends a hybrid architecture as the practical solution.

1. Introduction

Low-code platforms are very effective for workflow-centred business applications, and success with a small application often encourages organizations to use the same platform for much larger systems. This is where limitations appear. A case study helps show which requirements push a system beyond what a low-code platform handles comfortably, and how to respond.

2. Case Scenario

The organization in this scenario is fictional and the case is created for academic study. StayWell Hotels Group is a hotel chain with 250 properties in several countries. After a Room Booking application built in Zoho Creator worked well at one budget hotel, management proposes to extend it into a group-wide central reservation and revenue management system with these requirements:

- Real-time room inventory synchronized across all properties and online travel agencies.
- Thousands of concurrent booking requests in peak season with no double bookings.
- Dynamic pricing driven by machine learning demand forecasts.
- Integration with legacy property management, payment, loyalty and accounting systems.
- Strict compliance with data protection, payment security and audit rules in each country.

3. Analysis of Limitations

The requirements are examined against the typical limits of a low-code platform.

Area	Limitation on a Low-Code Platform	Impact on the Use Case
Performance and scalability	Scripts run under execution-time and API-call limits; fine-grained query tuning and caching are not available	Peak booking bursts and large inventory recalculations may be slow or fail
Concurrency and transactions	Limited control over transactions and locking	Risk of double bookings under simultaneous requests

Area	Limitation on a Low-Code Platform	Impact on the Use Case
Complex algorithms	Scripting suits business rules, not heavy computation or ML inference	Dynamic pricing must be built outside the platform
Integration depth	Connectors and REST cover common cases; legacy protocols and event streaming are hard	Custom middleware becomes necessary
Data volume	Storage and reporting are tuned for business-scale data	Slow reports; external data warehouse needed
UI customization	Limited to the components the builder offers	Brand-specific booking journeys are hard to achieve
Vendor lock-in	Logic, data model and workflows depend on the vendor runtime	Costly migration if pricing or features change
Governance at scale	Many makers and apps can cause sprawl and inconsistent security	Compliance and audit risk across countries
Testing and DevOps	Limited automated testing, version control and CI/CD	Harder to enforce quality gates in a large team

The most serious risks are concurrency and performance. A booking system must never sell the same room twice, and this needs close control over transactions and locking that a low-code runtime does not fully expose. Dynamic pricing and large-scale analytics are computation-heavy and fall outside what a business-rule scripting language is designed to do.

4. Mitigation Strategies

- Adopt a hybrid architecture: keep forms, workflows, approvals and dashboards in low-code, and move high-load or algorithmic parts to pro-code services.
- Use an integration layer such as an API gateway between the low-code application and legacy systems.
- Move large-scale analytics to a data warehouse.
- Establish governance with role-based access, separate development, staging and production environments, naming standards and central review.
- Run a proof of concept and load tests before committing the whole enterprise to a platform.
- Plan an exit strategy by keeping data exportable and business logic documented.

5. Recommended Decision

A full migration of the central reservation engine to a purely low-code platform is not recommended. The low-code application should be kept for front-office workflows, guest request forms and reporting at each property, while the real-time inventory and pricing engine is built as pro-code services connected through APIs.

Component	Recommended Approach
Guest request forms, front-office reports, approvals	Low-code (Zoho Creator)
Real-time inventory engine	Pro-code service exposed through an API
Dynamic pricing model	Pro-code or ML service (for example on Zoho Catalyst)
Group-wide analytics	External data warehouse and BI tool
Legacy system connectivity	Integration layer or API gateway

6. Conclusion

Low-code platforms are excellent for workflow-centred applications but are not a universal solution. Highly concurrent, algorithm-heavy and deeply integrated enterprise systems expose limits in performance, control and portability. Recognizing these limits early and adopting a hybrid architecture lets an organization keep the speed of low-code without giving up reliability.

7. Summary

The case of StayWell Hotels Group shows that a low-code application that succeeds at small scale cannot simply be stretched to an enterprise reservation system. Limits in performance, concurrency, algorithms, integration depth, data volume, customization, lock-in, governance and DevOps all apply. A hybrid design, supported by governance and load testing, gives the best balance.

8. Learning Outcome

This case study develops the ability to judge whether a low-code platform is suitable for a given enterprise requirement, to recognize its technical and organizational limits, and to propose a hybrid architecture that combines low-code with pro-code extensions.